



Exercício: Sobrescrita de Método com a Anotação `@Override`

Objetivo:

Demonstrar o uso correto da anotação `@Override` para sobrescrever métodos em classes Java e entender os problemas que podem surgir sem ela.

Instruções:

1. Crie uma classe chamada `SuperClasse` com um método chamado `imprime()`. O método deve imprimir a mensagem "imprime".
2. Crie uma classe chamada `MinhaClasse` que estende a `SuperClasse`. Sobrescreva o método `imprime()` na `MinhaClasse` para que ele imprima a mensagem "imprime diferente".
3. Certifique-se de usar a anotação `@Override` ao sobrescrever o método na `MinhaClasse`.
4. Na `SuperClasse`, adicione um comentário ao método `imprime()` explicando que ele será sobrescrito na classe filha.

Passos para fazer o exercício:

1. Defina a classe `SuperClasse` com o método `imprime()`.
2. Crie a classe `MinhaClasse` que estende `SuperClasse` e sobrescreva o método `imprime()`.
3. Utilize a anotação `@Override` na declaração do método sobrescrito.
4. No método `main`, crie uma instância de `MinhaClasse` e chame o método `imprime()`.
5. Compile o código **também** após comentar o método `imprime()` na classe pai para ver o erro, depois compile sem comentá-lo para deixar tudo funcionando corretamente.

Código do exercício:

```
public class SuperClasse {  
    /**  
     * Imprime a mensagem "imprime".  
     * Este método será sobrescrito na classe filha.  
     */  
    public void imprime() { // Você deve comentar esse método para  
        // ver o erro de compilação (para testar a anotação @Override).  
        System.out.println("imprime");  
    }  
}  
  
public class MinhaClasse extends SuperClasse {  
    @Override  
    public void imprime() {  
        System.out.println("imprime diferente");  
    }  
  
    public static void main(String[] args) {  
        MinhaClasse minhaInstancia = new MinhaClasse();  
        minhaInstancia.imprime();  
    }  
}
```

Explicação do código:

- A classe `SuperClasse` possui um método `imprime()` que imprime a mensagem "imprime". O comentário no método indica que ele será sobrescrito na classe filha.
- A classe `MinhaClasse` estende `SuperClasse` e sobreescreve o método `imprime()`, alterando a mensagem para "imprime diferente".
- O uso da anotação `@Override` garante que estamos realmente sobreescrevendo o método da classe pai.
- No método `main`, criamos uma instância de `MinhaClasse` e chamamos o método `imprime()` para testar o comportamento correto.



Exercício: Criando Métodos Depreciados

Objetivo

O objetivo deste exercício é praticar a criação e utilização de métodos em Java, além de entender o uso da anotação `@Deprecated`.

Instruções

1. Crie uma classe chamada `MinhaClasse` com dois métodos: `calcularSoma` e `calcularProduto`.
2. Utilize a anotação `@Deprecated` no método `calcularProduto` para indicar que ele não deve ser mais utilizado.
3. Crie uma classe chamada `Principal` com o método `main` para testar os métodos da classe `MinhaClasse`.
4. Compile e execute o programa para verificar os resultados.

Passos para Fazer o Exercício

1. Criação da Classe `MinhaClasse`:

- Defina a classe `MinhaClasse`.
- Implemente o método `calcularSoma` que recebe dois inteiros e retorna a soma deles.
- Implemente o método `calcularProduto` que recebe dois inteiros e retorna o produto deles, utilizando a anotação `@Deprecated`.

2. Criação da Classe `Principal`:

- Defina a classe `Principal`.
- No método `main`, crie uma instância de `MinhaClasse`.
- Chame os métodos `calcularSoma` e `calcularProduto` com valores de exemplo.
- Imprima os resultados no console.

3. Compilação e Execução:

- Compile o código utilizando um compilador Java.
- Execute o programa e verifique os resultados impressos no console.

Código do Exercício

```
public class MinhaClasse {
    public int calcularSoma(int a, int b) {
        return a + b;
    }

    @Deprecated
    public int calcularProduto(int a, int b) {
        return a * b;
    }
}

public class Principal {
    public static void main(String[] args) {
        MinhaClasse minhaClasse = new MinhaClasse();
        int resultadoSoma = minhaClasse.calcularSoma(5, 3);
        int resultadoProduto = minhaClasse.calcularProduto(5, 3);

        System.out.println("Soma: " + resultadoSoma);
        System.out.println("Produto: " + resultadoProduto);
    }
}
```

Explicação do Código

- **Classe `MinhaClasse`:**

- O método `calcularSoma` recebe dois parâmetros inteiros `a` e `b` e retorna a soma deles.
- O método `calcularProduto` também recebe dois parâmetros inteiros `a` e `b` e retorna o produto deles. Este método está marcado com a anotação `@Deprecated`, indicando que ele não deve ser mais utilizado em novos códigos.

- **Classe `Principal`:**

- No método `main`, uma instância de `MinhaClasse` é criada.
- Os métodos `calcularSoma` e `calcularProduto` são chamados com os valores `5` e `3`.
- Os resultados são impressos no console.

Este exercício ajuda a entender como criar e utilizar métodos em Java, além de demonstrar o uso da anotação `@Deprecated` para marcar métodos que não devem ser mais utilizados.



Exercício Complementar: Suprimindo Warnings com a Anotação `@SuppressWarnings`

Objetivo

O objetivo deste exercício é entender como usar a anotação `@Deprecated` e suprimir avisos relacionados a métodos obsoletos.

Instruções

1. Crie uma classe chamada `ExemploDepreciado`.
2. Dentro dessa classe, defina um método chamado `metodoObsoleto()`. Anote esse método com `@Deprecated`.
3. Implemente outra classe chamada `Principal`.
4. No método `main` da classe `Principal`, crie uma instância de `ExemploDepreciado` e chame o método `metodoObsoleto()`.
5. Suprima o aviso do compilador relacionado à anotação `@Deprecated`.

Passos para fazer o exercício

1. Crie um novo projeto Java.
2. Implemente as classes `ExemploDepreciado` e `Principal` conforme as instruções acima.
3. Compile e execute o código.

Código do exercício

```
// Classe ExemploDepreciado
public class ExemploDepreciado {
    @Deprecated
    public void metodoObsoleto() {
        System.out.println("Este método está obsoleto.");
    }
}

// Classe Principal
public class Principal {
    @SuppressWarnings("deprecation")
    public static void main(String[] args) {
        ExemploDepreciado exemplo = new ExemploDepreciado();
        exemplo.metodoObsoleto(); // Suprimindo o aviso do compilador
    }
}
```

Explicação do código

- A classe `ExemploDepreciado` contém um método chamado `metodoObsoleto()`, que foi marcado como obsoleto usando a anotação `@Deprecated`.
- Na classe `Principal`, usamos a anotação `@SuppressWarnings("deprecation")` para suprimir o aviso do compilador relacionado à chamada do método obsoleto.